

PRUEBA TÉCNICA · CIBERSEGURIDAD

Decisiones de Arquitectura (ADR)

Registro de decisiones clave

PREPARADO POR	Lady Marcela Romero Rivero
ROL	Líder de Ciberseguridad
CLIENTE	Simon Movilidad
ESCENARIO	FleetSec S.A.S. — Telemetría de flotas
FECHA	25 de agosto de 2026
VERSIÓN	1.0

CONFIDENCIAL — Exclusivo para Simon Movilidad

Tabla de contenido

ADR-0001 — Stack: Node.js 20 + Express, own minimal vulnerable app

Context

Decision

Consequences

ADR-0002 — DevSecOps quality gates and thresholds

Context

Decision

Consequences

ADR-0003 — Terraform security baseline & documented Checkov exceptions

Context

Decision

Consequences

ADR-0004 — Detection engineering & incident response approach

Context

Decision

Consequences

ADR-0005 — Handling the "traps" embedded in the test requirements

Context

Decision (trap → resolution)

Consequences

Registro de las decisiones técnicas clave del programa de seguridad FleetSec.

ADR-0001 — Stack: Node.js 20 + Express, own minimal vulnerable app

• **Status:** Accepted · **Date:** 2026-08-23 · **Deciders:** Security Engineering

Context

The test allows a free stack and either a known vulnerable app (Juice Shop, DVWA) or a minimal own build for the VAPT. We must demonstrate the 10 specific CWE classes and remediate **≥8 in our own code** with paired malicious/legitimate tests.

Decision

Build a minimal Express (Node 20) app with a `/vuln` lab surface (the 10 CWE) and a `/secure` remediated twin. Use pure-JS dependencies (`sql.js` WASM SQLite, no native build) to keep the Docker image slim and the pipeline fast.

Consequences

- (+) We control the exact 10 CWE; every PoC and every fix is reproducible and unit-tested (20/20).
- (+) Remediation lives in our code (rubric rewards this over patching third-party apps).
- (+) `docker compose up` is a single fast command (Bonus +5%).
- (–) A hand-built lab must be transparently documented (e.g. the XXE parser is deliberately naive) — noted in README and the finding.
- (–) Not a "famous" target; mitigated by mapping every finding to OWASP/CWE and offering Juice Shop as an optional comparison bank.

ADR-0002 — DevSecOps quality gates and thresholds

• **Status:** Accepted · **Date:** 2026-08-23 · **Deciders:** Security Engineering

Context

The pipeline must enforce hard gates (SAST, SCA, DAST, container, IaC, secrets) yet the repo intentionally contains a vulnerable lab. A naive "0 findings" gate would either fail permanently or force us to delete the teaching material.

Decision

Two-tier SAST: an **informational** Semgrep run over the whole repo (shows the custom rules firing, non-blocking) plus a **blocking** gate over production code only (`app/src` , excluding `vulnerable.js` and tests), with inline `nosemgrep` suppressions carrying `razón · fecha · responsable` for the V-10 lab secret. Thresholds: SCA blocks HIGH/CRITICAL ($\approx$ CVSS ≥ 8 direct / ≥ 9 indirect), container blocks CRITICAL, DAST HIGH/CRITICAL block + MEDIUM opens a `security/medium` issue, Checkov hard-fails (built-in CKV_AWS_24/25 enforce the admin-port rule). Independent jobs run in parallel with npm/Trivy caches to stay ≤ 15 min.

Consequences

- (+) Gate reflects real production risk; suppressions are auditable (mirrors the required SAST discipline).
- (+) Custom rules are still demonstrably exercised.
- (+) Parallel fan-out + caching meets the ≤ 15 -min SLA.
- (–) The exclusion list must be reviewed so it never hides a real production finding — owned by SecOps in CODEOWNERS.

ADR-0003 — Terraform security baseline & documented Checkov exceptions

- **Status:** Accepted · **Date:** 2026-08-23 · **Deciders:** Security Engineering / Cloud Platform

Context

Entregable 03 asks for a `modules/security-baseline` module modelling a secure AWS foundation, with `terraform validate` clean and a Checkov scan. Some requested settings are actually **traps** (see ADR-0005) and some Checkov checks are out of scope for a networking/data baseline (e.g. the ALB/ECS resources live in the app stack).

Decision

Author a self-contained module (VPC 3-tier, KMS CMKs, S3 + Object Lock, RDS Multi-AZ, Secrets Manager, CloudTrail, Config, GuardDuty, Security Hub, WAFv2) whose SGs are consumed by the app stack via outputs. Drive Checkov to **0 failed** by fixing every cheap real gap (default-SG lockdown, access logging, RDS IAM auth/PI/enhanced monitoring, lifecycle, CloudTrail SNS, WAF logging) and adding **12 inline documented skips** (`checkov:skip=ID:reason`) for intentional/out-of-scope items (e.g. ALB port 80 redirect per requirement, cross-region replication accepted risk, KMS root key policy). Add a custom guardrail (CKV2_FLEETSEC_1 YAML + unit-tested Python).

Consequences

- (+) `checkov` result is audit-grade: 251 passed / 0 failed / 12 documented skips.

- (+) The admin-port trap is enforced by built-ins (CKV_AWS_24/25) that PASS our SGs and FAIL a bad SG.
- (–) Full `terraform validate / plan` should be run locally (no cloud creds in CI); HCL syntax validated offline for all 12 files.
- (–) draw.io artifact is generated from the provided Mermaid source.

ADR-0004 — Detection engineering & incident response approach

• **Status:** Accepted · **Date:** 2026-08-23 · **Deciders:** Security Engineering

Context

Entregable 04 provides a fully-embedded breach timeline. We must produce IOCs, detections, an IR playbook, MITRE mapping, RCA and executive comms.

Decision

Prioritise detections high on the **Pyramid of Pain** (behaviours/TTPs over IPs): 4 Sigma rules (IAM admin off-hours, DeleteTrail anti-forensics CRITICAL, SQLi in app logs, bulk S3 GetObject). Load IOCs into a GuardDuty **Threat Intel Set** via Terraform (preferred) with an AWS CLI fallback. IR follows NIST 800-61 (preserve→contain→eradicate→recover→lessons) with exact reversible AWS CLI, MITRE ATT&CK v14 (8 techniques), 5-Whys + Swiss-cheese RCA, a ≤1-page CEO brief, and the Ley 1581 SIC notification path (15 business days).

Consequences

- (+) Rules validated with `pysigma` (parse-clean, 4/4). A DeleteTrail *attempt* alerts even when SCP blocks it.
- (+) Threat Intel Set is version-controlled and reproducible.
- (–) Sigma aggregation syntax (bulk GetObject) is backend-specific — documented; translate with `sigma convert`.
- (–) IP-based IOCs (185.220.101.22) age fast — re-evaluate every 90 days; behavioural rules carry the load.

ADR-0005 — Handling the "traps" embedded in the test requirements

• **Status:** Accepted · **Date:** 2026-08-23 · **Deciders:** Security Engineering

Context

Several requirements are deliberately worded to tempt an insecure or non-compliant implementation. A senior engineer must detect them and correct rather than comply blindly. This ADR records each trap and the professional decision taken. Full analysis in [README.md](#) §"Análisis crítico y trampas".

Decision (trap → resolution)

1. **KMS "key policy sin Principal AWS *"** — a wildcard `Principal: "*"` on a key policy is dangerous. We used the account-root admin statement + scoped service principals (never `Principal: "*"`). See [kms.tf](#).
2. **SG "0.0.0.0/0 excepto 80/443 en ALB"** — only the ALB SG exposes 0.0.0.0/0, only on 80 (redirect) and 443; admin ports never. Enforced by `CKV_AWS_24/25`.
3. **RDS `ssl=1` / `log_connections=1`** — implemented via a parameter group (`rds.force_ssl=1` , `log_connections=1`), plus IAM auth and no public endpoint.
4. **V-10 "migrar a variable de entorno o gestor de secretos — nunca mover a otro archivo del repo"** — moving a secret to another repo file is the trap; we removed it to env/Secrets Manager. See [secure.js](#) / [rds.tf](#).
5. **DAST "cobertura ≥80% del OpenAPI"** — we ship an OpenAPI spec of the *secure* surface so ZAP has a real contract to authenticate against.
6. **Break-glass** — must be audited: 2 reviewers via a protected Environment + an auto-opened CRITICAL issue on every override (`break-glass.yml`).
7. **AI report** — must **admit** at least one AI hallucination; hiding it scores zero. Documented honestly in [docs/ai-report.md](#).
8. **`svc-monitoring` service account with console login** — service accounts must not have console access; the IR plan revokes it and the P1 backlog adds an SCP.

Consequences

- (+) Demonstrates security judgment over literal compliance — the core of the evaluation.
- (+) Each correction is traceable to code/evidence.
- (–) A reviewer expecting literal compliance may need the rationale — hence this ADR and the README section.